

- A_0 algorithm
 - Optimal algorithm (similar to OS)
 - A_0 is optimal under the independent reference model.
 - LRU-K gives results close to this algorithm!

→ Burmired K-distance

- Given a reference string known upto time t , $r_1, \dots, r_t$, the burmired K-distance is the distance burmired to the k^{th} most recent reference to the pg. p :
- Set K-distance to ∞ if an item is not referenced $\geq k$ times
- Remove the page with the longest K-distance from memory to replace
 - If multiple pages have same maximum, use classic LRU as a tie breaker

→ Problems

- Early Page Replacement: Four ways in which a pair of references might access same disk pg:

→ Intra-transaction: Read record in page, then update record

→ Transaction-Retry: Access pg; Abort: Access pg again

→ Intra-process: Access initial records on pg; Commit:

→ Inter-process: Independent access by multiple transactions

- Interarrival times must be calculated based on non-correlated access pairs!

→ Each successive access by the same process within a time out period (Correlated Ref. period) are taken to be correlated, and hence will be considered as a single reference.

DBA timing set this value to 5 seconds

→ Retained information

- Each time a pg. is referenced, it is made buffer resident.

- If estimate of $b_k(p, n) = \infty$, it is the prime candidate victim. What if the pg. is popular, but just not referenced k times.

- Hence, we need to maintain history information about any pg. for some period after its most recent access.

Set to 200s (3min 20s) Retained information period (RIP)

→ Maintain addl. data structure to solve these problems

- HIST(p)

→ Represents history window of pg. p

→ Contains times of 2 most recent uncorrelated references to p

→ $LAST(p)$

→ Denotes the time of most recent reference to page p .

→ The Data Structures can be stored in memory. They can be stored in pg. header on disk, that means that we will be dirtying each read pg., even if we're not updating data.

→ Monitored for all pgs. with backward $K < RHP$

→ Pseudo code:

```
if p is already in the buffer
then /* update history information of p */
  if  $t - LAST(p) > Correlated\_Reference\_Period$ 
  then /* a new, uncorrelated reference */
     $correl\_period\_of\_refd\_page := LAST(p) - HIST(p,1)$ 
    for  $i := 2$  to  $K$  do
       $HIST(p,i) := HIST(p,i-1) +$ 
         $correl\_period\_of\_refd\_page$ 
    od
     $HIST(p,1) := t$ 
     $LAST(p) := t$ 
  else /* a correlated reference */
     $LAST(p) := t$ 
  fi
else /* select replacement victim */
   $min := t$ 
  for all pages  $q$  in the buffer do
    if  $t - LAST(q) > Correlated\_Reference\_Period$ 
      /* if eligible for replacement */
      and  $HIST(q,K) < min$ 
      /* and max Backward K-distance so far */
    then
       $victim := q$ 
       $min := HIST(q,K)$ 
    fi
  od
  if victim is dirty then
    write victim back into the database fi
  /* now fetch the referenced page */
  fetch  $p$  into the buffer frame previously held by victim
  if  $HIST(p)$  does not exist
  then /* initialize history control block */
    allocate  $HIST(p)$ 
    for  $i := 2$  to  $K$  do  $HIST(p,i) := 0$  od
  else
    for  $i := 2$  to  $K$  do  $HIST(p,i) := HIST(p,i-1)$  od
  fi
   $HIST(p,1) := t$ 
   $LAST(p) := t$ 
fi
```

clean pages are prepared
usually to avoid this

→ Optimal value of K

→ There is a minor improvement from $K=2$ to $K=3$ however there is a huge improvement from $K=1$ to $K=2$.

→ LRU-3 consumes much more resources. Hence LRU-2 is the "sweet spot".

→ Mid-Term (30 marks)

→ Room 117, 11/10

→ Exam starts @ 8:30, ends @ 2 pm

→ Printed papers on 13/10, 8:30 - 7:30 pm

→ Question Types

→ Working out

→ Derivation

→ What-if (E.g. What if we do REDO-undo in ARIES)

→ Design extension (E.g. Lock compatibility matrix & acquisition protocol w/ new lock mode)

→ Open book

→ Bring own hard-copies of papers/slides/addl. materials

→ Laptop recharging allowed

no network connectivity

Distributed Databases

13/10/25 (M)

→ Advantages:

→ Availability → Parallelism

→ Locality → Load balancing

→ Reliability → Scalability

→ Autonomy
A server can behave independently of other servers.

→ Disadvantages:

→ More complex to maintain & program

→ Communication costs (network latency) needs to be accounted for

→ Design goal: From a user's perspective, the distributed system should appear like a normal system.

→ Simple case: $R \bowtie S$, R is @ A, S is @ B, and user is @ C

→ Options

→ Ship R & S to C and choose local strategy

→ Ship R to B, compute the join & send to C

→ " S " A, " " " " " " " "

→ Semi-join Strategy:

→ Compute $\Pi_S(R)$ at A & ship to B.

→ Compute $S \bowtie \Pi_S(R)$ and ship to A.

→ Compute $R \bowtie (S \bowtie \Pi_S(R))$ and ship to C

This must be equivalent to $R \bowtie S$

→ Proof of correctness: $R \bowtie (S \bowtie \Pi_S(R))$

$$= (R \bowtie \Pi_S(R)) \bowtie S$$

$$= R \bowtie S$$

→ Cost estimates: Assume communication cost = $C_1 + C_2 d$, where d = no. of bytes transmitted.

Assume disk I/O cost = $C_3 d$

→ N_R = No. of tuples in a reln. R

→ S_R = " " bytes " " tuples of reln. R .

→ $V_{X,R}$ = " " distinct values of attribute X in R

$$d = N_R S_R$$

→ Semi-join can be useful in a centralised system.

→ Centralised need not mean "one system". It can be two servers present in the same location.

→ Guaranteeing ACID

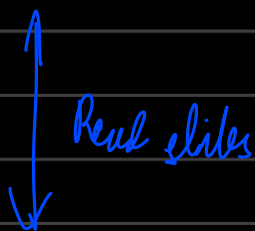
→ Each site can be locally ACID-compliant. However local acid $\neq$ global acid

→ All sites may implement the same transaction. Therefore, we need a commit protocol to ensure uniform decision across all sites participating in the distributed execution of the transaction.

→ 2PC (2-Phase Commit) protocol: guarantees global atomicity

→ Master sends START WORK messages to cohorts along with the associated work.

→ A cohort after completing will send a WORK DONE message to master



→ Choice of K:

→ Large value: less chance of blocking

→ Small " : " delay in normal processing

Concurrency Control in Distributed Systems

→ To maintain global concurrency (serial order).

→ If all sites use basic 2PC, then can some problem occur?

→ Yes. Two individual databases may choose different serial schedules:

D_1 can choose $T_1 \rightarrow T_2$, and

D_2 " " $T_2 \rightarrow T_1$

abort / commit

do not release X locks until end of transaction

→ Soln: Use strict 2PC

→ This ensures concurrency, but this assumes that there is only one copy of data. i.e. there is no replication of data.

→ If replication is used, there are numerous techniques for concurrency control: "read only, write all" / "primary copy" / "majority vote"...

used with most transactions are read only

used when transactions are 50/50 in read/write

* Recommended reading: Concurrency control & recovery in database systems
P. Bernstein, V. Hadzilucas & V. Hadzilucas
Addison-Wesley, 1987

Lock Release Time

prepared to commit,
ready message sent

- Read locks @ cohort can be released when transaction goes into prepared state.
- However, write locks need to be held until completion of transaction, i.e. master sends commit abort confirmation.
- If master sends abort, undo the transaction before releasing exclusive locks.
- Otherwise, some other transaction may quickly acquire the lock and read the corrupt values.

Centralised Handling

- A global wait-for graph is present in the master node.
- If there is a cycle in the graph, there is a deadlock.
- Disadvantage: The local lock managers will have to communicate with the master extensively. Even if the transactions are disjoint, they will still have to send the data.

Fully Distributed Handling

- Each site has a local wait-for graph, the system combines these graphs to obtain a combined wait-for graph.

↑
Read Loc. slides
↓

- Phantom deadlocks: A dead lock victim is chosen, even though the deadlock may have disappeared.
- This occurs due to delay between deadlock detection & deadlock solution. ↪ Maybe the transaction aborted by itself.

Mid-term Statistics

22/10/25

Avg: 16.5 People b/t 20-25: 8 ⇒ Approx. rank = 7/23

Special class: 2pm, Friday, CSA-112

Project presentations: 9:30 onwards, CSA-252

Data Mining

Data mining: Automated & efficient extraction of "interesting" statistical models from enormous disk-resident archival databases. ↪ or patterns

↪ upto 10^{15} bytes

Interesting patterns:

- Associations: Capture object attribute relationships
E.g. If student is from Delhi, high probability of "MADE EASY" coaching.
- Clustering: Group similar objects together
E.g. Google groups webpages (with similar content) together.
- Classification: Assign objects to categories low/mid/high risk
E.g. Vehicle insurance categories are decided based on owner's age & vehicle colour.
- Deviations: Detect "abnormal" behaviour
E.g. To detect cheating / doping in sports & olympics / match fixing.

- Benefits of data mining:
 - Better business strategies: Action movies released in July succeed in box office.
 - Improved customer services: "Frequently bought together"

* Privacy-preserving data mining: How to modify i/p data s.t. statistical model does not change, but i/p cannot be traced back.

Association Rules

→ Co-occurrence of events

→ E.g.: 80% of customers who bought coffee bought milk also:

Not a functional dependency, functional dependency will have 100%, because it is schematic.

Coffee $\Rightarrow$ Milk (0.8)

Depiction of above statement

→ To ensure significance of data, we need to compute the "support" also:

E.g. Coffee & milk are bought together by 60% percent of customers

X & Y must be disjoint

→ Problem Formulation: Given the data, find all rules: $X \Rightarrow Y$ (c, s), where c = min-confidence and s = min-support.

$$c = P(Y|X), \quad s = P(X \cup Y)$$

hard to calculate

1) Find all itemsets I which satisfy min-confidence requirement

2) Use frequent data sets to generate "rules"

→ Simple soln.

→ The set of all itemsets is a lattice, i.e. $(2^n - 1)$ sets. Find these itemsets &

→ Done using a one-pass algo, by incrementing the count for each item set present in each tuple.

→ Results in no. of counters = 2^m , $m = |I|$

- In a supermarket, most item sets shall not meet the required min-confidence. We shall be doing unnecessary computations.
- Apriori algorithm can be used to solve this problem.

Apriori Algorithm

→ Procedure:

- Multiple scans over the complete database.
- In scan i :
 - Read database row-by-row
 - Count occurrences of candidate item sets of length i
 - At the end of each scan, determine the frequent item sets of length i : F_i
 - Determine the "candidate" item sets for $(i+1)^{th}$ iteration
- Return $F_1 \cup F_2 \dots \cup F_m$

E.g.: First Pass: Read database row-by-row
 Count occurrences of all individual items
 (counters are stored in 1-D array)
 Determine F_1 (items which match min-support)
 " candidate item sets (with length 2) for the next scan by
 $F_1 \times F_1$

1-item sets

Second pass: Count occurrences of 2-item sets by storing counters in a 2-D LTM
 w/o diagonal
 (not full array because the matrix is symmetric, diagonals are also not reqd.)
 Determine F_2
 Determine candidate item sets for next scan using Apriori-Gen algorithm

Apriori Gen Algorithm

→ To generate candidates C_k , we use 2 phases:

→ Join phase: $F_{k-1} \times F_{k-1}$

Insert into C_k :

Select $p.i_1, \dots, q.i_{k-1}$

From $F_{k-1} p, F_{k-1} q$

Where $p.i_1 = q.i_1, \dots, p.i_{k-1} = q.i_{k-1}$ and $p.i_k < q.i_k$

To preserve lexicographic order

→ Pruning phase

lec. notes

* Monotonicity property: Any subset of a frequent itemset must be frequent on its own.

Same as before, except candidate itemsets are stored in a hash-tree. Non-leaf nodes contain a hash table, with each entry in the bucket pointing to another node at the next level. Leaf nodes contain a list of itemsets.

Subset Search

To check all candidate subsets of transaction T ,

lec. notes

→ Summary:

- No. of passes is proportional to longest frequent itemset.
- Works for sparse matrices

Vertical Mining

24/10/25

Data layout options: → Row-wise or coln. wise?
→ Full data or bitmap? ↗
presence only both presence & absence

Apriori uses row-wise & full data.

Vertical mining uses coln-wise & bitmap.

Why vertical organization?

- Natural for association rule mining's goal of discovering correlated items.
- Support counting shall be simpler.
- No excess baggage from disk.
 - For-e.g., when we tune row-wise we shall be reading rows which have a low support.
- Ideal for parallel implementation, because they are not synchronous.
 - Each column can be calculated individually, without having to wait.
 - Even if apply parallel programming on apriori algorithm, we shall have to wait until everyone stops counting.

Why bit vector representation?

- Compression can be done, because of small alphabet size.
- Despite the fact that we are storing presence AND absence, we shall save on storage after compression (it will be worse before compression).

→ Beats the "optimal" in some cases!!



VIPER Algorithm

→ First pass of VIPER

- Snakes are created from the original horizontal database.
- Compression using "winning" algorithm, based on holomb encoding.
- Compute F_i

Despite most DBs being vertical, we do this to accommodate for the worst case.

* R.L.E. does not work very well here because the database is sparse. The ones are mostly isolated. R.L.E shall end up doubling the size of the database. This shall be worse than the apriori database because we are adding "absence bits" also.

holomb encoding

→ holomb encoding basically stores the no. of occurrences only.

* To ensure holomb encoding, we just add a 1 in the beginning.

HW: Write code for holomb encoding for better understanding.

→ $w_0 = 256 / 512$ and $w_i = 1$

```

Algorithm VIPER( $\mathcal{D}$ ,  $\mathcal{I}$ ,  $minSup$ ){
  Input: Horizontal Database ( $\mathcal{D}$ ), Set of items ( $\mathcal{I}$ ),
        Minimum Support ( $minSup$ )
  Output: Set of Frequent Itemsets with Supports ( $F$ )

  // Pass 1: Write all 1-snakes to disk and identify  $F_1$ 
   $F = \text{countLevelOne}(\mathcal{D})$ ;
  // Pass 2: Identify  $F_2$ 
   $F = F \cup \text{countPairs}(F_1)$ ;
  // Subsequent Passes
   $i = 2$ ; until ( $\text{isEmpty}(F_i)$ ) {
    // Create a new DAG for this level
     $candDAG = \text{createDAG}(\text{level} = i)$ ;
     $C_i = F_i$ ;
    // Candidate Generation for levels  $k+1$  to  $2k$ 
    for  $k$  in  $i$  to  $2i$  do {
       $C_{k+1} = \text{FORC}(C_k)$ ;
      if ( $\text{isEmpty}(C_{k+1})$ ) break;
       $candDAG = candDAG \cup C_{k+1}$ ;
    }
    if ( $\text{isEmpty}(candDAG)$ ) break; // Terminate

    // List of snakes to be read
     $readList[i] = \text{findReadList}(candDAG)$ ;
    // Trim the list of snakes to be written
     $writeList[i] = \text{writePrune}(candDAG)$ ;

    // Snake intersection, counting and writing
     $\text{FANGS}(candDAG, readList[i], writeList[i])$ ;

    // Update  $F$  from levels  $i+1$  to  $2i$ 
    for  $k$  in  $i+1$  to  $2i$  do
       $F = F \cup \text{frequentItems}(candDAG, k)$ ;

    // Delete the snakes written in previous pass
     $\text{DeleteSnakes}(writeList[i/2])$ ;

    // Increment the mining level
     $i = i * 2$ ;
  }
}

```

Figure 2: The VIPER Algorithm

→ Second pass of VIPER

→ Compute F_2 by intersecting all "snake" intersection, essentially requiring $|F_1| C_2$ set intersections.

→ Hence, we create a streaming horizontal database is created in memory. All pairs of items are found as per the apriori algorithm, however, we do not write F_2 to disk.

- Snake writing
 - writing all candidate snakes to disk is very expensive & redundant, since many turn out to be infrequent.
 - Introduce a lag of one pass in writing snakes. Hence, only frequent snakes are written to disk.
 - The "unwritten" snakes shall be required as inputs to the current pass. Hence, we regenerate them dynamically.



- Subsequent passes of VIPER using FORC algorithm
 - generates candidates from levels $(i+1)$ to $(2i)$, based on equivalence class clustering.

- Snake intersection using FAVAS algorithm



Performance Metrics

- ORACLE algorithm
 - Ideal algorithm which knows in advance which tuples are "popular".

Data Lakes

- Data warehousing: Consolidate data from many sources in one large repo.
 - Loading & periodic synchronisation from replicas.
 - Multiple sources may have multiple titles & meaning, thereby requiring semantic integration.

- OLAP

- complex SQL queries & views, with queries based on "spreadsheet"-style operations.
- Needs to be interactive

→ Data warehousing expects quick work over petabytes of data.

→ Semantic integration: While getting data from multiple sources, we must eliminate mismatches, i.e. different currencies / schemas.

→ Heterogeneous sources: Must access data from a variety of source formats & repositories

→ Load, Refresh, Purge: We must load data & refresh it periodically, and keep purging old data.

→ Multidimensional data model: Collection of numeric measures, which depend on a set of dimensions.

→ MOLAP vs. ROLAP

→ MOLAP: multi-dimensional OLAP

→ Multi-dimensional data is stored physically in a disk-resident, persistent array.

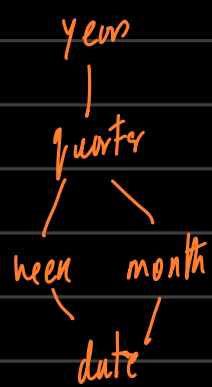
→ ROLAP: relational OLAP

→ We use relational systems.

→ We have a main reln. that relates dimensions to a measure.

→ Each dimension can have addl. attributes and an associated dimension table.

→ Dimension hierarchies: For each dimension, the set of values can be organised in a hierarchy:



fact table

→ OLAP queries:

→ Inspired from SQL & spreadsheets

→ A common operation is to aggregate a measure over one or more dimensions.

→ Roll-up: Aggregating based on levels of hierarchy.

E.g. Total sales by country or total sales by state / city.

- Drill down: Reverse of roll-up
- Pivoting: Aggregation on selected dimensions.
- Slicing: Equality selections on one or more dimensions
- Dicing: < Slide >
- CUBE operator:
 - If there are K dimensions, we can have 2^K possible SQL GROUP BY queries.
- CUBE tables: Uses ALL to represent the set over which aggregation is computed.
 - * Technically a violation of relational model, but works!?
- CUBE Research issues: ✓
 - Storage structures - ROLAP / MOLAP? → Index structures for cubes?
 - Schemas for ROLAP - star / snowflake? → Level of materialisation of cube.
- Star schema:

< Add diagram from slides >

- Fact table is in BCNF, but dimensional tables are not normalised
 - * BCNF is used because it removes 100% of data redundancy.
- Dimension tables are not normalised to move it fast. Normalisation slows down performance.
- Furthermore, dimension tables are much smaller than fact tables. These tables are also relatively static.
- Computing joins of such schemas, is called as a star join.
- Snowflake schema:
 - Dimension tables are also normalised to reduce redundancy.
 - to some extent only, not BCNF.



→ (UBE indexes:

- Joins are common b/t fact & dimension tables
 - Indexes are req'd. for interactive responses.
 - Space is a problem
- * Updating the index is not an issue due the tables being static (mostly)

→ Most companies have two databases for OLTP & OLAP. OLTP is migrated to OLAP periodically.
→ We can combine these into a sort of 'OL*P' database, such as in S V.

→ Such DBs will have lots of user-defined functions (UDFs). However, such UDFs are treated as a black box from query optimizer perspective.

(UBE Materialisation

→ Only way to give quick responses is to pre-compute.

→ Issues in view materialisation:

- What views to materialise, and which indexes to build on this view?
- Given a query & materialised views, can we re-write the query in terms of materialised views.
- How frequently do we refresh the materialised views? How to incrementally do this?

E.g. we have $R_0 \bowtie S_0$, what to when $R \rightarrow R_0 \Delta R$ and $S \rightarrow S_0 \pm \Delta S$

$$\text{Hence } (R \bowtie S) = (R_0 + \Delta R) \bowtie (S_0 + \Delta S)$$

$$= \underbrace{R_0 \bowtie S_0}_{\text{already calculated}} + \underbrace{\Delta R \bowtie S_0 \pm R_0 \bowtie \Delta S}_{\text{very difficult to calculate}} + \underbrace{\Delta R \bowtie \Delta S}_{\text{relatively simple to calculate}}$$

→ Given N dimensions, we have 2^N views in lattice

→ Options

- Materialise every thing
- Min. response time
- Space explosion

- Materialise nothing
- No addl. space reqd.
- High response time

→ Correct option: Materialise a subset & derive other views from it.

→ Given a view lattice & a constraint of the no. of views that can be materialised, which choice would result in min. org. cost?

→ Assumptions

- All queries equi-probable
- Query cost $\propto$ no. of rows examined
- No indexes

→ Optimisation problem is NP-Hard, hence use greedy:

Algo: V = view lattice

$S = \{\text{top view}\}$ // top view must be materialised because all "sub-views" can be generated from it.

for $i = 1 \rightarrow n$ do:

do a full traversal of $V - S$

select view $v \in S$ s.t. $\text{benefit}(v, S)$ is maximised

$S = S \cup \{v\}$

Benefit computation: $\text{benefit}(v, S)$:

For each $w \leq v$ // view w can be derived from v

Let there be a view u s.t. $u \in S$ and $w \leq u$ and u is the least cost in S that can derive w

If $C(v) < C(u)$, then $B_w = C(u) - C(v)$

Else $B_w = 0$

Overall benefit = $\sum_{w \leq v} B_w$

→ Reference: $\frac{B_{greedy}}{B_{opt}} \geq 1 - \left(\frac{n-1}{n}\right)^n \geq 0.63$ (Tight bound)

As $n \rightarrow \infty$, $\frac{B_{greedy}}{B_{opt}} \rightarrow 0.63$

When $n = 2$, $\frac{B_{greedy}}{B_{opt}} = 0.75$

Some randomised algos. can do better, but they are probabilistic

→ No better algorithm is possible!

→ Special cases

→ Close to optimal if first view delivers most of the benefit

→ Equal " "

→ Alternative problem formulation

→ Total space is fixed, rather than no. of views to materialise

→ Benefit per unit space (R.O.I.) needs to be computed.

In-Memory Databases

→ Move from memory management to cache management.

→ Cache vs. main mem. buffers:

→ Everything in memory will be software-controlled, cache is like a black box

→ Caches may have set associativity, which is why we may have cache miss even with an almost-empty cache.

→ Cache may separate based on instruction & data (I-cache & D-cache).

→ Commit trade (PV cycles for improving cache performance (time in linear hashing paper).

→ types of miss:
 → compulsory → Capacity (Miss even in FA cache) → Conflict (would not have missed in FA cache)

→ Locality Techniques
 → blocking (instead of nested tuple join, use nested block join)
 → Partitioning (line in external sorting)
 → Data filtering / clustering
 → Loop fusion

→ Data Structures
 → Vertical Decomposition
 → binary association table: one per column of original reln. BAT
 → Each tuple of BAT has (OID, value) object ID
 → OID can be implicit, virtual OID or VOID

→ Join processing

→ Partitioned joins
 → cluster both relations.
 → create clusters that fit in cache memory
 → Join matching clusters
 → Radix join (partitioned nested loop)
 → Partitioned hash join

"Item" Table

supplier	part	price	discount	qty	tax	flag	status	shipmode	date1	date2	comment	
•	10	•	04.75	•	•	0.10	•	•	AIR	•	•	•
•	10	•	11.50	•	•	0.00	•	•	MAIL	•	•	•
•	11	•	10.20	•	•	0.00	•	•	TRUCK	•	•	•
•	11	•	75.00	•	•	0.00	•	•	AIR	•	•	•
•	11	•	02.50	•	•	0.00	•	•	SHIP	•	•	•
•	12	•	92.80	•	•	0.10	•	•	AIR	•	•	•
•	13	•	37.50	•	•	0.10	•	•	SHIP	•	•	•
•	13	•	14.25	•	•	0.00	•	•	MAIL	•	•	•

int int int float float int float char(1) int varchar date date date char(27)

width of relational tuple ≈ 80 bytes

vertical fragmentation in Monet

oid	int
1000	10
1001	10
1002	11
1003	11
1004	11
1005	12
1006	13
1007	13

oid	float
1000	04.75
1001	11.50
1002	10.20
1003	75.00
1004	02.50
1005	92.80
1006	37.50
1007	14.25

oid	float
1000	0.10
1001	0.00
1002	0.00
1003	0.00
1004	0.00
1005	0.10
1006	0.10
1007	0.00

oid	str
1000	AIR
1001	MAIL
1002	TRUCK
1003	AIR
1004	SHIP
1005	AIR
1006	SHIP
1007	MAIL

void	chr
	-2
	-3
	-1
	-2
	-6
	-2
	-6
	-3

encoding BAT

chr	str
0	REG AIR
1	TRUCK
2	AIR
3	MAIL
4	RAIL
5	FOB
6	SHIP

physical data structures

logical appearance

8 bytes

8 bytes

8 bytes

optimized BAT storage: 1 byte per column

→ Advantages:

- Variable no. of keys per node
- Fewer levels and lesser space reqd.
- Speed increased by 20%.

→ CSB (cache sensitive B) trees:

- Balanced multinary search tree
- Tree of order d contains m keys, where $d \leq m \leq 2d$
- A child nodes of a node are stored in one group.
- Instead of $m+1$ pointers, we will use only 1 pointer

to the first child

→ Node structure

→ Node size = cache line



→ Node search techniques

→ Basic approach: binary search with while loop

→ Code expansion approach:

→ Loop-unfolding using if-then else

→ Pad unused slot with highest possible key

no counter with the key

→ Uniform approach

→ More binary unfolding right-deep

→ Single traversal function.

similar to Huffman encoding

→ Variable approach:

→ Traversal function is dependant on key cardinality

→ Offline compute optimal traversal function

→ Index appropriate function at runtime. (using hOTOs)



→ Deletion

→ Done lazily, i.e. index entry is not deleted, but rather marked with a flag.

→ As cache block size increases, cost of deletion will be higher.

→ Ex: cache line size of 128 B

→ Segmented B+ Trees

→ Divide child nodes into segments, with nodes in each segment are stored contiguously.

→ Store pointers to each segment in parent node.

→ Search is slower

→ Full B+ trees

→ Pre-allocate space for a full node group * lesser malloc calls, saves more time.

→ cheaper to copy in case of a split

→ less memory allocation overhead (triggered by node group overflow)

→ Requires 30% more space

→ Results: < See lec. slides for the full comparison >

→ If enough space, choose full B+ trees

→ If search > update, " variants of B+ trees.

Columnar Databases (C-STORE)

less reads,
more writes

→ Row stores are write optimised, i.e. they are suitable for OLTP.

→ What about OLAP?

→ Mostly read ops. (occasional writes to bulk load new data)

→ Very few relevant cols. in ad-hoc queries - (only 10% on avg.)

→ Row store is not optimal for such applications.

- Advantages of column stores:
 - Fetch only req'd. cols. → Better cache effects
 - Compression is better due to data being homogeneous.

because

- Disadvantages:
 - Includes extra joins due to tuple reconstruction of cols. from some table
 - Requires more space for tuple identifiers
 - Can be slow for OLTP, due to multiple columns (which may be stored in different locations) needing to be updated.

C-STORE ← data-heavy approach

- Data model (storage)
 - Only materialised views → Innovative redundancy
 - Compress columns to save redundancy
- Columns are stored as projections. (A set of columns, which are pre-computed & stored)

- Projections are horizontally partitioned into "segments". Each segment has:
 - Segment identifier → Unit of distribution & parallelism
 - Value-based partitioning, key range of sort keys.

- Storage keys
 - Within a segment, every data value of every column is associated with a unique storage key.
 - Values from different columns with matching storage key belong to the same logical row.

* Virtual in Read store, explicit in Write store

- Reconstructing the base table from projection
 - Projections are joined using storage keys & join indexes
 - Join Index

↑
↓
Slabs (17)

- Needs multiple join indexes for reconstruction of base table
- Join indexes are costly to maintain
- Therefore, move each coln. part of several projections

→ Compression (use different schemes for each column, depending on ordering & values)

→ Self-order, few distinct values: (value, position, num-entries)

E.g. [4, 12, 7] // Row no. $12 - (2+7) = 19$ is all 4s

Clustered B-tree on value to position

RLB

→ Foreign order, few distinct values: (value, bitmap)

Clustered B-tree on position to value

→ Self-order, many distinct values: block oriented delta encoding

E.g. 1, 4, 7, 7, 8, 12 $\Rightarrow$ 1, +3, +3, +0, +1, +4

Clustered B-tree on block starting values

→ Foreign order, many distinct values: do not encode

Unclustered B-tree on values

→ Redundant Storage

→ want house rarely recovered from log // time too long

→ Storage enough projection s.t. k -safety is ensured // even if we lose x projections, other projections must be able to recover these projections

→ Rebuild lost projections from elsewhere in the network

→ Transactional updates

→ Necessary even for mostly read databases due to:

→ Clerical errors

→ Real-time data warehouses

→ Batch updates of data

Read stores 20 onwords
(till final)

- Avoid long updates
 - Optimise read-only queries; do not hold locks
 - Snapshot isolation
 - Query is run on a snapshot of the data as of effective time (ET)
 - Ensure transactions related to snapshot have committed



- Query optimisation (new operators)
 - Decompress to get back original data
 - Concatenate: If multiple columns are in the same order, we just combine them

Final Overview

22/11 - (SA 11.7)

8:30 am - 2:00 pm

Topics 11 - 19 (post mid-sem)

30 minutes

Distributed Databases : QP / TM

Data Mining : Apriori & Viper

" Warehousing : Data cube & data views

Main memory DBMS : cache-conscious joins & embeddings

Columnar " : L-store & coarsening (indexes)

Columnar indexes for Row Reconstruction

← lazy approach

- How to make tuple reconstruction easier?
 - If columns are in sorted order, we can just join the two columns together.
- Database coarsening
 - Self-tuning database kernel

← Example of automatic computing

* Any time algorithm: If we stop at an arbitrary time, will I get some

output? Need not be optimal output or correct output, but does an answer exist?

- if we build indexes w/o knowing the workload, we may end up doing wrong indexes!!!
- Each query is treated as an "advice" on how data should be stored. Physical re-org. happens per column based on selection predicates.
- The first time an attribute is required by a query, a copy of coln. A is created, called as the warmer column C_A .
- Each selection operator on A triggers a range-based physical reorganization of C_A .
- Each warmer coln. has an AVL tree (covering index) to maintain partitions

Why are AVL trees used instead of B-trees?

- warmers . select (A, v_1, v_2)
 - create C_A if it does not exist
 - Searches the index of C_A for the ones where $v_1 \leq v_2$ fail
 - if the bounds do not exist, no query used them in the past

slide 9-15

- Adaptive alignment
 - Extend the sideways select operator

Queries are not stored / flushed into disk. When the system shuts down, the indexes are simply discarded. If the workload has changed after restart, we are better off reconstructing the index.

